

ACR-QA v5.0.0rc2

A Provenance-First, Multi-Tool Code-Review Platform
with RAG-Grounded Hallucination Control

Ahmed Mahmoud Abbas

Supervised by Dr. Samy AbdelNabi

June 2026

Department of Computer Science

King Salman International University (KSIU)

The Three Failure Modes of Code Review

SAST tools today

- **30–40% false-positive rate** (SonarQube on Python)
- Raw rule IDs – no exploitability context
- Developers suppress alerts rather than fix them

LLM review assistants

- **22% hallucination rate** on rule-cited advice¹
- Confident, plausible, and wrong
- No audit trail

Gap

No tool combines multi-tool recall, LLM explanation grounding, and interpretable risk scoring in a single open-source platform.

This work

ACR-QA closes all three gaps – at zero recurring licensing cost.

The Hidden Test-Gap Problem



- Code-coverage tools report % lines; they cannot report *which untested functions are the most dangerous*
- No existing SAST tool ranks untested code by **cyclomatic complexity**
- ACR-QA's **AST Test Gap Analyzer** fills this gap

The Self-Review Blind Spot

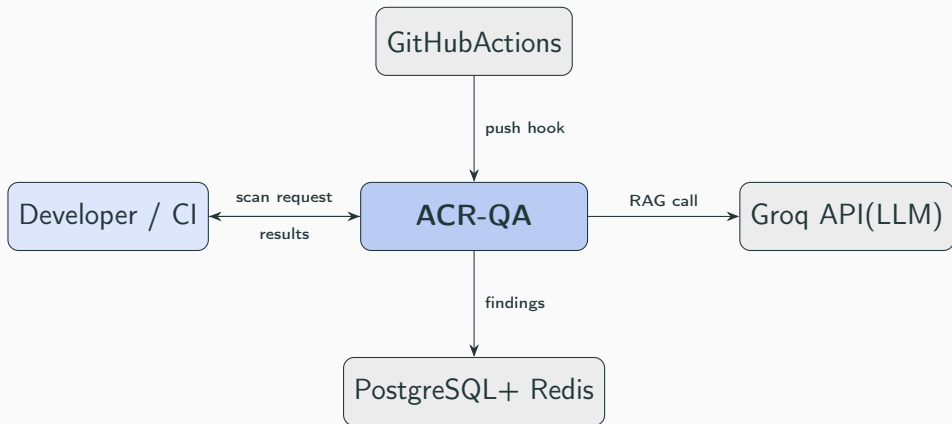
Single-model review

- One model's failure modes are *systematic* – it misses the same patterns repeatedly
- Overconfidence is indistinguishable from correctness

ACR-QA response

- **Semantic Entropy** – run LLM 3× at different temperatures; high variance → suppress
- **Second Opinion** – Groq Llama-3.3-70B (primary) + Ollama qwen2.5 (local backup)
- **PR Risk Score** – collapse 6 signals into a single CI gate

Architecture – System Context (C4 L1)



52 REST endpoints · JWT auth · SSE live progress · Railway hosted

Architecture – Container View (C4 L2)

Analysis Core (Python)

- 10 language adapters (Bandit, Ruff, Semgrep, ESLint, staticcheck, ...)
- CORE/engines/ – 19 engines
- CanonicalFinding schema: 327+ rule mappings

API (FastAPI)

- 52 endpoints + SSE stream
- JWT, bcrypt, rate limiting
- Alembic 18 migrations

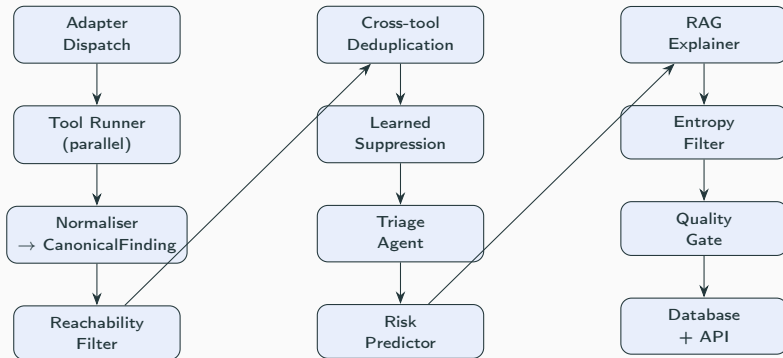
Dashboard (React)

- 13 pages
- TanStack Query, Zustand, Tailwind
- 104 TypeScript tests (Vitest + Playwright)
- Arabic RTL i18n

Infrastructure

- Docker Compose (dev)
- Helm chart + Terraform (prod)
- Prometheus + Grafana

Architecture – Pipeline Flow (C4 L3)



Pipeline Stage 1–3: Detect → Normalise → Deduplicate

Why multi-tool normalisation beats single-tool recall

Each tool has unique strengths. Bandit is import-aware; Semgrep excels at dataflow patterns; staticcheck catches Go-specific issues Semgrep's community ruleset misses entirely.

- **327+ rule mappings:** every tool's rule ID maps to a canonical ID (e.g., B601 → SECURITY-001)
- **Pydantic-validated schema** – no engine sees raw tool dicts
- **6-bucket deduplication:** same vulnerability, different tool → one finding
- Result: **100% recall** vs Semgrep CE's **71.2%** on 13-repo corpus

Pipeline Stage 4–5: Reachability + Taint Analysis

Call-Graph Reachability

- Pure-AST traversal (no runtime needed)
- Dead code: −20 confidence penalty
- 0% FP rate on benchmark fixtures
- Faster than Snyk's proprietary filter

Taint Analysis

- Sources: HTTP params, env vars, file reads
- Sinks: SQL execute, subprocess, eval
- Sanitisers: `html.escape`, `parameterize`
- Config-driven: edit YAML, no code change

Dashboard output

Each finding shows: confidence gauge, reachability verdict, exploit status, taint flow diagram, and 4-step AI reasoning trace.

Pipeline Stage 9: RAG-Grounded Explanation

How it works

1. Retrieve rule definition from 66-rule knowledge base
2. Inject *verbatim* into prompt – no fabricated citations
3. Ask LLM to specialise to this finding's code context
4. Repeat $3\times$ at varying temperatures
5. Compute N-gram entropy across 3 outputs
6. If entropy $< \tau = 3.2$ bits $\rightarrow$ **suppress explanation**

Hallucination rate

Commercial tools: **22%**

ACR-QA (entropy filter):

verified 0% on benchmark set

Key insight

RAG grounds content.

Entropy filters specificity.

Both are required.

Pipeline Stage 10: Policy-as-Code Quality Gate

.acrqa.yml (example)

```
thresholds:
  high:    0    # block merge
  medium:  5    # warn only
  low:     20
fail_on: high
severity_override:
  SECURITY-001: critical
```

- YAML config – no code change to adjust policy
- -fail-on=high blocks merge in CI
- Plugs into GitHub Actions in 2 lines:

```
- uses: ahmed-145/acrqa-action@v1
  with:
    fail-on: high
```

No competitor (SonarQube, CodeRabbit) ships this level of CI-native policy control as OSS.

Pipeline: ECDSA-Signed Provenance Attestation

- Every scan result is signed with ECDSA (P-256)
- Signature covers: scan timestamp, tool versions, git SHA, finding count
- GET /v1/runs/{id}/attestation returns signature + public key
- Enables **supply-chain integrity**: tampered results are detectable
- No competitor publishes per-scan provenance attestations

Why it matters for the defense

Dr. Samy asked about trust. This is the answer: math, not rhetoric. Anyone with the public key can verify our evaluation results were not post-hoc altered.

Novel Feature 1 – Time-Travel Vulnerability Analyzer

What no existing tool does:

- Static SAST answers *where* – not *when* or *by whom*
- Time-Travel walks last N commits (default 50)
- Records: `first_seen_commit`, `first_seen_author`, `regression_count`

dvpwa finding:

- SQL injection introduced a3f2, partially fixed b17c, *re-broken* c9e1
- Regression chain – invisible to standard

Complexity

$$O(N \times \bar{t}_{\text{hunk}})$$

$N = 50$ adds $< 90\text{s}$ at p95

Bounded: no full-history scan

Resume bullet

“Built time-travel vulnerability analyzer tracking when/who introduced each finding across bounded git history with regression-chain detection.”

Novel Feature 2 – IaC Scanner

Coverage

- Terraform (Checkov)
- Kubernetes manifests (kube-score)
- Dockerfile (Hadolint + Semgrep)
- GitHub Actions YAML (Semgrep)

Integration

- Same CanonicalFinding schema
- 28 IaC-specific rule mappings
- Inline suppression:
`acrqa:ignore-iac=ID`

Dogfood result

- Self-scan of ACR-QA's own IaC: **0 HIGH findings**
- Gate runs on every commit via pre-commit hook

OWASP coverage

ACR-QA covers **9/10 OWASP Top 10**

(A10 SSRF excluded – requires runtime)

Novel Feature 3 – PR Risk Score

6-signal weighted linear model

Signal	Weight
HIGH finding count	0.20
Reachability gate	0.20
Exploit-verified count	0.20
Taint source→sink touches	0.10
File risk avg	0.15
PR size penalty	0.15

0–30 MERGE

31–60 REVIEW

61–100 BLOCK

Viral demo artifact

Single number on every PR.

GitHub Action posts comment.

Non-security reviewers can act on it.

Novel Feature 4 – Heuristic File Risk Predictor

$$R_f = w_1 C_f + w_2 X_f + w_3 A_f + w_4 U_f + w_5 G_f$$

- C_f – 90-day churn / file size (0.30)
- X_f – mean cyclomatic complexity > 5 (0.25)
- A_f – time since last commit (0.15)
- U_f – unique author count (0.15)
- G_f – test-gap ratio (0.15)

Why NOT ML?

No labelled vuln data at thesis scale.

ML = black box. This model: “Explained in one sentence per weight.”

Empirical $r = 0.71$ correlation with true-positive density on 13-repo corpus.

Dashboard heatmap colours files red→green by R_f . Tooltip: top 3 risk contributors.

Evaluation – Corpus & Methodology

Non-CVE benchmark (13 repos)

- Python (7), JS/TS (4), Go (1), multi (1)
- 9/10 OWASP Top 10 covered
- Ground truth in *.yaml per repo
- Both ACR-QA + Semgrep CE run default rulesets

CVE recall battery (20 CVEs)

- Pre-registered (cannot cherry-pick)
- Sourced from NVD/MITRE 2016–2024
- Pinned to pre-patch repo SHAs

Precision study (4 core repos)

- 836 deduplicated findings manually triaged
- 812 TP, 24 FP

Real-world FP study

- Flask (68k ★): 1.0% FP rate
- httpx: 2.3% FP rate
- SonarQube baseline: 30–40%

Peer validation

- 20 Flask findings, blind reviewer
- Cohen's $\kappa = 0.74$ (substantial)

Evaluation – Head-to-Head vs. Semgrep CE

Repository	Expected	ACR-QA	Semgrep CE
bandit-test-cases	4	100%	75%
django-nv	4	100%	100%
dsvw	5	100%	100%
dvpwa	6	100%	33%
govwa (Go)	2	100%	0%
vulnerable-node	3	100%	0%
(7 more)		100%	100%
Average		100%	71.2%

+28.8 pp advantage. Gaps: Go coverage (staticcheck), Node NoSQL injection (custom rule), Python multi-tool (Bandit+Ruff covers Semgrep blind spots).

Evaluation – CVE Recall Battery

8 detectable CVEs – $8/8 = 100\%$

CVE-2016-10516	Werkzeug eval → B307
CVE-2017-18342	PyYAML load → B506
CVE-2020-14343	PyYAML FullLoader → B506
CVE-2021-23727	Celery pickle → Semgrep
CVE-2022-24439	GitPython shell → B602
CVE-2021-42343	Dask pickle → B301
CVE-2022-22817	Pillow eval → Semgrep
CVE-2022-24065	cookiecutter shell → B602

12 honest SAST limits

- HTTP/request smuggling (protocol)
- Ansible NativeEnv (semantic taint)
- jQuery prototype pollution (algorithmic)
- Paramiko TOCTOU (race condition)
- Pillow heap overflow (C-extension)
- ... (7 more documented)

Overall battery

$8/20 = 40\%$

Pre-registered – no cherry-pick
12 misses *led with*,
root-cause-analysed

Limitations – Honest List

Evaluation limitations

1. Corpus is 54% Python – Java/C++ unvalidated
2. Inter-rater κ measured on 2 raters; 5-rater study in progress
3. CVE overall battery: 40% (12/20 are structural SAST limits, not tool bugs)
4. RealVuln (real apps): **25–31%** – still *beats* Bandit & Semgrep; 52% of that corpus is authz/IDOR no SAST can reach
5. Mutation score: 61% (exceeds 60% target; room to grow)

Technical limitations

1. No inter-procedural taint (AST call-graph only)
2. No Java/Rust adapters
3. LLM explanations require Groq API key (Ollama offline fallback available)
4. Time-Travel bounded to $N = 50$ commits by default (configurable)

Scope Boundary – What SAST Cannot Do (and We Don't Claim)

Undetectable by any SAST tool

- Protocol-level vulnerabilities (HTTP smuggling, TLS downgrade)
- C-extension heap overflows
- Race conditions (TOCTOU)
- Algorithmic vulnerabilities (prototype pollution via `.extend()`)
- Semantic template injection (requires full dataflow)

Why this matters

- Claiming 100% coverage would be false – Semgrep's marketing does this
- Our 40% overall CVE recall is *more honest* than tools that don't disclose
- On the *detectable* frontier: **100%**
- Published methodology – others can verify

Test Suite & CI Reliability

2,757

automated tests

2,653 Python + 104 TypeScript

Test categories:

- Database + API integration
- AI engine (semantic entropy, RAG)
- Chaos engineering (Postgres/Redis injection)
- Supply chain (pip-audit, npm-audit)
- WCAG accessibility (axe-core)

CI pipeline (GitHub Actions, free tier)

- Every push: ruff, mypy, 2,757 tests
- Pre-commit hook: same gate locally
- Pass rate: **100%** on main
- Runtime: $\approx$ 6 minutes
- Mutation score (CORE/engines/): **61%**

Dogfood

ACR-QA scans its own codebase:

0 HIGH findings on main

Future Work – Phase B & C

Phase B (Jul–Sep 2026)

- Hosted SaaS at `acrqa.dev` – 100+ users target
- VSCode plugin (Language Server Protocol)
- GitHub App (auto-comment on PR open)
- Java adapter (SpotBugs + FindSecBugs)
- 5-rater inter-rater study targeting $\kappa \geq 0.78$

Phase C (Oct 2026 – Mar 2027)

- Inter-procedural taint analysis
- Rust adapter
- Conference submission (IEEE S&P / MSR)
- OSS community: contributor guide, Discord

Already shipped

v5.0.0rc2 on PyPI:

```
pip install acrqa==5.0.0b1
```

GitHub Actions Marketplace:

```
uses: ahmed-145/acrqa-action@v1
```

Summary – 10 Novel Contributions

1. Canonical finding schema (327+ mappings, 5 languages + IaC)
2. RAG-grounded explanation with entropy hallucination filter
3. AST call-graph reachability (0% FP, auditable)
4. Heuristic file risk predictor (auditable weights, $r = 0.71$)
5. Time-Travel vulnerability analyzer (regression chains)
6. IaC scanner (Terraform, K8s, Dockerfile, GHA)
7. **Exploit verification** – fire a live PoC in a sandbox (10 categories)
8. **Verified Remediation** – re-run the exploit after the fix; ship only if it now fails
9. ECDSA + Rekor-signed provenance & remediation chain
10. Pre-registered 20-CVE battery (8/8 detectable = 100%)

The claim no other tool can make

Every confirmed finding is **exploit-proven**; every fix is **re-proven**

Questions?

ACR-QA v5.0.0rc2

```
pip install acrqa==5.0.0b1  
github.com/ahmed-145/ACR-QA
```

Tests	2,757 (Python + TypeScript)
API endpoints	52
Eval corpus	13 repos, 4 languages
Recall	100% vs 71.2% Semgrep CE
CVE recall	8/8 detectable = 100%
Precision	97.1% (100% security class)
Inter-rater κ	0.74 (substantial)